

Delta Lake Rust Developer Guide

Tips and Recommendations for Developers Who Want to Write Code in Rust with the Delta Lake Protocol

1. What is Delta Lake?

Delta Lake is an open-source storage format that helps you work with big data in a safe way. It keeps a log of every change you make, like a notebook where you write down everything you do with your data. The Rust library is called `delta-rs`. It lets you use Delta Lake without Java or Spark. Rust is very fast and uses less memory. The repository is on GitHub at `delta-io/delta-rs`.

2. How Does Delta Lake Work?

Delta Lake keeps your data in two parts. The transaction log is inside a folder called `_delta_log`. Every time you change the table, a new numbered JSON file is added. The data files are Parquet files (columnar format, fast to read). Delta Lake also creates checkpoints - summary files that help the system load data faster without reading every small log file.

Concept	What It Means	Where You Find It
Transaction Log	Record of all changes	<code>_delta_log/</code> folder
Checkpoint	Summary file for faster loading	<code>.checkpoint.parquet</code>
Parquet Files	Actual data in columns	Partition folders
Actions	Add, Remove, Metadata operations	JSON commit files
Snapshot	Table state at a version	In memory after loading

3. The Rust Project Structure

The project has many small crates. The most important is `deltalake-core` - the heart with table management, reading, writing, and operations. Other crates add cloud storage support (AWS, Azure, GCP). When you start, you only need the main `deltalake` crate.

Crate	What It Does
<code>deltalake</code> (main)	The main package - use this one to start
<code>deltalake-core</code>	Table management, reading, writing, operations
<code>deltalake-aws</code> / <code>azure</code> / <code>gcp</code>	Support for cloud storage backends

4. How to Start Your First Project

Add the library to your `Cargo.toml`. You need Rust edition 2024 (version 1.91.1+). The library uses async code with tokio runtime:

```
# Cargo.toml
[dependencies]
deltalake = { version = "0.32", features = ["datafusion"] }
tokio = { version = "1", features = ["full"] }

// main.rs
use deltalake::open_table;
fn main() {
    let rt = tokio::runtime::Runtime::new().unwrap();
    rt.block_on(async {
        let table = open_table("./my_table").await.unwrap();
        println!("Version: {:?}", table.version());
    });
}
```

5. Important APIs You Should Know

The `DeltaTable` struct is the main object. Build it with `DeltaTableBuilder` to set options like version or buffer size. All operations (create, write, merge, update, delete, vacuum, optimize, restore) are methods on `DeltaTable`.

Method	What It Does	Example
<code>open_table(url)</code>	Open a table	<code>open_table("./data").await</code>
<code>table.create()</code>	Create new table	<code>.with_columns(cols).await</code>

table.write(batches)	Write data	.with_save_mode(Append)
table.merge()	Merge into target	.when_matched_update()
table.scan_table()	Read all data	Returns stream of batches
table.load_version(v)	Time travel	table.load_version(5).await
table.optimize()	Compact files	.with_compaction().await

6. Key Rust Patterns Used in Delta-rs

The codebase uses many Rust best practices. Understanding these will help you read the source code and write better code.

Async / Await: Almost every operation is async. Use .await after each call. The library uses tokio. All loading, reading, writing, and committing are async - good for slow storage operations.

Builder Pattern: Used everywhere. DeltaTableBuilder for opening tables, operation builders for create/write/merge. Chain methods like .with_columns(), .with_partition_columns(), .with_save_mode().

Traits for Abstraction: LogStore trait for interchangeable storage (S3, Azure, local). DeltaWriter for writing. Operation trait for all table operations. Clean design, easy to extend.

Error Handling: Uses thiserror crate. All errors in DeltaTableError enum (20+ types). Use ? to propagate errors, or .unwrap() only in examples.

7. Important Tips for Developers

- 1. Choose one TLS backend: RustTLS is the default. Use "native-tls" feature only if needed. You cannot have both at the same time.
- 2. Do not use DeltaOps: The old DeltaOps(table) wrapper is deprecated. Use methods directly on DeltaTable (e.g., table.create() not DeltaOps(table).create()).
- 3. without_files() saves memory: If your app only appends data and never reads files, use .without_files() when building the table.
- 4. Tune log buffer size: Default is 4 x CPU cores. Reduce this if your cloud storage has rate limits.
- 5. Enable only needed features: "datafusion" for SQL queries, "rustls" for TLS. Extra features make compile time longer.
- 6. Read the actions: Each commit has Add, Remove, Metadata actions. Understanding these helps you understand how Delta Lake tracks changes.